

Devolución TP1

Grupo 1

Demo

- Detalle de restaurante no cambia según sección
- <http://pawserver.it.itba.edu.ar/paw-2025a-01/restaurants/1/reservations/create> no tiene favicon
- El ancho del contenido es inconsistente (<http://pawserver.it.itba.edu.ar/paw-2025a-01/restaurants/29/turn-templates/tab-turns> vs <http://pawserver.it.itba.edu.ar/paw-2025a-01/restaurants/29/tab-info>)
- La creación de turnos falla en ciertos casos (puede ser al pasar medianoche)
- Link al crear reserva (para ver desde el perfil de comensal) es erróneo
- Intentar XSS en creación de menú tira 500
 - Inhabilita la edición de menú
- No limitan cantidad de reviews por persona ni si fue o no al restaurant
- No hay recuperación de contraseña
- Bueno:
 - Catchean tipo de archivo ante subida de imagen
 - Bueno el link de maps
 - Precio promedio se saca de reviews
 - Flujo simple para la gestión de turnos pero se podría unir al manejo de horarios

Código

- Definen versión de dependencias en pom hijo (a veces con variables definidas en el padre, a veces directo con el número de versión)

- Uso de dependencia de spring-boot (cuando en realidad, también usan la dependencia de mailing correcta)
- Guardan en el repositorio archivos de configuración del editor de texto (.vscode) y de configuración de mvn con archivos vacíos (.mvn)
- Usan `${pageContext.request.contextPath}/` en vez de `<c:url>`
- Loggean a nivel debug en producción
- Tienen un archivo `schema.sql` a nivel webapp, cuando esto concierne a la capa de persistencia.
- Validan un form a mano en el controller en vez de usar un custom validator
- Hacen llamados a varios services al crear un restaurant para crear el modelo y para cargar la imagen, cuando debería ser un único método al que se le encargue toda la creación y que este llame a los otros servicios
- No protegen los endpoints de `TurnTemplateController`
- Tienen restricciones en la creación de reservas que validan en capa de servicios pero no en los forms, lo que lleva a que hagan catch de excepciones para poder mostrar mensajes de error
- Utilizan el string `http://pawserver.it.itba.edu.ar/paw-2025a-01/restaurants/`, lo cual se puede desacoplar con una variable de entorno
- El Locale que usan al enviar mails de reviews al dueño del restaurant es el del cliente
- En varias instancias hacen uso de una lista de strings para los días, la cual debería reemplazarse por un enum
- Falta cobertura de tests para los servicios con validaciones complejas
- Para el menú, mezclan usos de null, Empty y un objeto instanciado con id negativo para indicar inexistencia de un dato
- El orderBy debería ser un enum más que un magic string
- En vez de hacer que el DAO devuelva el rango horario ya en la forma "HH:mm - HH:mm", tendría más sentido tratarlo como un par de tiempos hasta el momento de mostrarlo
- Hay oportunidades de reutilización de código en los filtros de restaurante

- La creación de DTOs se mejoraría con builders o métodos `.fromModel`, si bien no parece ser necesario el uso de los mismos.

Seguimiento

- Ok seguimiento, ok planificación

Nota

- 7

Grupo 2

Demo

- El registro está raro... "registro mi empresa", para caer a una pantalla que me bloquea, donde para seguir tengo que clicar en un botón "registrar mi empresa"
- En el alta de insumo no están creando bien el HTML de `optgroup` s
- Al ir a un proceso > Planificación > Crear Planificación la pantalla me habla de "tareas". Al querer crear una tarea, da un 500 (sólo pasa si no hay tareas previas)
- Más que editar inventario, debiéramos procesar despachos de la misma. Sólo al hacer un inventario una empresa quiere "pisar" el valor, y así como está hoy esto hasta genera condiciones de carrera, ya que en background los procesos pueden estar modificando el inventario.
- La creación de un proceso y su schedule no está guiado, no hay un workflow claro.
- Los textos de i18n de error no están bien, la aplicación muestra mensajes como `El nombre del proceso debe tener hasta processName caracteres`

Código

- No siguen la convención de nombres de Java, con clases como `Relation_Task_TaskSchedule`
- El código hace abuso de magic strings, que son repetidos y conocidos a lo largo y ancho de la aplicación, atravesando múltiples capas de la misma.

- Clases como el `ScheduleServiceImpl` son extremadamente complejas, denotando una pobre modularización y modelos anémicos.
- Hacen uso de clases de JUnit 3.
- Los tests de persistencia nunca validan el estado de la base de datos. **Esto es un error grave.**

Seguimiento

- Definen las tareas sin definir el tipo (feature, chore, bug)

Nota

- 7

Grupo 3

Demo

- Hay oportunidad desaprovechada de flujos sin login
 - Ej: ejercitar un deck
 - Ver una comunidad
- Puedo llegar a un flujo sin salida si trato de crear un posteo en una comunidad sin antes haber creado un deck
- Mensajes default "email.duplicate" para errores en el registro.
- Hay un abuso de HTML5 para validaciones de front o modales de confirmación de acciones sin su contraparte en back
 - No usan verificaciones en el back para los required de las cards
 - Ni para límite
 - Falta errores también en crear comunidad
- No validan los queryParámetros en el search (ej: pag=-1)
- Cambié el idioma y no puedo acceder al perfil
- Edit de comunidad tira 500
- El delete tampoco anda tira 400

- La página tiene leves quirks de UX
 - doble scrollbar
 - botones que se ven a la mitad
 - La sidebar de filtros me tapa el feed y no me deja sortear los elementos
 - Los filtros no poseen la mejor selección de componentes, Para los tags estaría bueno un autocomplete y para los min max un slider.
- Los emails son muy básicos y poco interesantes (tienen solo 2 que avisan de cosas que hace el propio user que realiza la acción)
- Falta paginar seguidores
- Bien de features. El flujo de encontrar y ejercitar un deck está bien armado.

Código

- Muchísima lógica de negocio en controllers, **esto es un error conceptual grave**. Ej `viewCommunityFollowers`
- No se está haciendo buen uso de la base de datos, en vez de hacer `JOIN`, se itera y se obtiene recursos haciendo N accesos a la db, lo cual luego se nota en la velocidad de la página.
- Hay constantes en los controllers sin uso.
- Hay muy poco uso de bean validation. Los forms casi no tienen annotations y se hacen chequeos manuales donde se podrían haber hecho custom annotations.
- Muy buena cobertura de tests en servicios.
- Cobertura nula de tests en persistence, todos los archivos están comentados.
- Utilizan boxed-primitives sin hacer correcto uso de las mismas.
 - Además los getters y setters devuelven las primitivas (unboxing). Esto los expone a `NullPointerException`
- Utilizan el paquete `java.sql` en capa de modelos
- No siempre utilizan modificadores de acceso. ej: `CommunityPostWithUserData`
- No están utilizando `@Transactional( readOnly = true )` para las funciones que no modifican la db.

Seguimiento

- Ok de seguimiento ok el Plane

Nota

Entrega tardía 14 minutos

- 5

Grupo 4

Demo

- Cuando me registro no me llega ningún correo de verificación pero tampoco me deja logueado. Debería dejarme logueado si no hay mail de verificación
- Cuando creo un viaje la primer vez me desloguea y luego me pide loguearme
- Si elimino un viaje que es en menos de 24 hs se rompe
- Estaría bueno mostrar el puntaje del conductor en el search
- Hay una doble barra // en el search
- Puedo eliminar los viajes de otras personas
- Estaría bueno poder editar los detalles de un viaje
- No se muestran las tabs de otros estados en viajes a menos que tengan algún elemento, queda raro y sería mejor mostrarlos con algún mensaje de empty
- Estaría bueno un CTA en el mail de cancelled que te lleve al search con los filtros similares al viaje cancelado
- La cantidad de dígitos del review debería estar redondeado a 2
- Faltaría poder ver mis reviews

Código

- En el pom padre sería mejor que todas las versiones de las dependencias estén parametrizadas en lugar de que algunas de ellas estén hard coded.

- El archivo schema.sql debería estar en el modulo de persistence y no en el modulo de webapp
- Imprimen mensajes directamente sin utilizar `<c:out>` sin escapar el contenido, lo que los hace vulnerables a XSS. **Esto es un error grave.**
- Es innecesario tener que almacenar y utilizar la variable `webapp_base_url` en la capa de webapp para los JSP, `<c:url>` ya resuelve esto
- Poseen lógica de negocio en los controllers
- Hacer try catch de exception en los controllers no es correcto, estos errores deberían ser manejados con el Exception Handler y retornar el error correspondiente. Si desean hacer alguna validación de negocio se deben utilizar custom validators.
- Solo poseen validación de roles en las rutas, falta un nivel mas fino de control de accesos para evitar que un usuario pueda realizar acciones sobre recursos que no le pertenecen.
- En el `HomeServiceImpl` el método `parseMaxPrice` debería ser `private`. Hay mas lugares donde existen método públicos que deberían ser privados.
- Utilizan `LocaleContextHolder` para el envío de mails en lugar de utilizar el locale del destinatario.
- Hacen "Joins" en Java en lugar de utilizar el motor de base de datos, esto es muy ineficiente ya que implica realizar multiples consultas a la DB de manera innecesaria.
- `UserTripsData` no es modelo de su sistema, sino un wrapper de varios modelos del sistema.
- Es inconsistente el uso del `SecurityContextHolder`, a veces lo utilizan en webapp para obtener el user ID para pasarlo al servicio y otras veces lo utilizan directamente en el service.
- Solo utilizan `@Transactional` en 4 métodos, esto es incorrecto.
- Utilizan `Mockito.verify`, esto valida la implementación y no el comportamiento. **Esto es un error conceptual grave.**
- Utilizan boxed-primitives sin hacer correcto uso de las mismas.

Seguimiento

- Las Features están bien definidas
- Hay Chores que en realidad son features como "Hacer que el mail de aceptación de viaje redirija a la pestaña de mis viajes" ya que aportan valor al usuario.

Nota

- 6

Grupo 5

Demo

- Faltarían filtros como por ejemplo precio y puntuación.
- Tienen un sort "relevance" que no se basa en ningún parámetro relacionado a relevancia.
- Al aplicar la búsqueda por texto, en la search bar no se muestra el valor aplicado.
- Al cambiar contraseña debieran pedir la contraseña actual.
- El perfil está bien diseñado, sólo que no permite editar los campos una vez seteados.
- Falta feedback al realizar acciones.
- Estaría bueno añadir CTA en las cards cuando se muestran como lista de entidades.
- Si busco un freelancer que no existe, la aplicación muestra 403 en lugar de 404

Código

- Pushean archivos vacíos al repositorio como `jvm.config` y `maven.config`.
- Obtener el usuario autenticado (`((AuthUser) SecurityContextHolder.getContext().getAuthentication().getPrincipal()).getUser().getUserId();`) podría extraerse a un método o ControllerAdvice para evitar reescribirlo todo el tiempo.

- Implementan la verificación de usuario de manera manual, en lugar de utilizar la implementación provista por Spring Security. Ver parámetro `enabled` de `userdetails.User`.
- Validan control de acceso tanto desde `WebAuthConfig` como usando `@PreAuthorize` en los controllers. Debiera ser consistente y usar una forma u otra, preferiblemente en `WebAuthConfig`, ya que permite concentrar todo el ACL en un solo lugar.
- La lógica de páginas podría manejarse en una clase wrapper `Page<T>`, de esta forma se evita tener que repetir la fórmula para calcular por ejemplo la cantidad de páginas.
- Tienen diferentes métodos dependiendo de qué parámetros aplican. Lo adecuado sería tener un solo método que acepte parámetros opcionales
- Handlean exceptions de SQL en la capa de webapp y utilizan la clase `java.sql.Date`. La existencia de una base de datos es un detalle de implementación del DAO, no debiera verse reflejada en webapp.
- Loguean a salida estándar sin utilizar un logger adecuado. **Esto es un error grave.** Por ejemplo `exception.printStackTrace();`.
- Todos los logs de GlobalExceptionHandler loguean `"SQL Exception {0}"`
- Muestran mensajes de error sin internacionalizar.
- La anotación `@ExceptionHandler` soporta múltiples exceptions juntas, de esta manera no tienen que repetir el mismo código por cada exception similar.
- No validan los valores que reciben en los forms. **Esto es un error grave.**
- Definen parámetros como `jobStatus1` y `jobStatus2`, lo cual no es nada claro qué implica cada uno. Lo que además lleva a repetir código de manera innecesaria
- Utilizan boxes primitivos y no parece ser por diseño
- Definen clases que luego no son utilizadas
- No tiene mucho sentido que las exceptions customs reciban un mensaje como parámetro si luego van a usar el mismo mensaje genérico que no aporta más contexto.
- Utilizan `[]` en lugar de clases más adecuadas como `List`. Revisar item 28 de Effective Java

- Los métodos `create` de los services son inconsistentes donde algunos retornan la nueva entidad creada y otros son `void`.
- La convención para el nombre de los modelos es la forma singular. Por ejemplo `Users` debiera ser `User`.
- No usan el propósito de la clase `Optional` de manera correcta. Por ejemplo,

```
Users user = userDao.findByEmail(email).orElse(null);
if (user!=null &&...
```

- El método `resetPassword` en `UserServiceImpl`, además de intentar el reinicio de password, realiza la validación de usuario, lo cual parece no tener sentido o al menos no estar correctamente documentado.
- Tienen la URL de la aplicación hardcodeada en la capa de servicios en lugar de ser inyectada apropiadamente mediante un archivo de properties.
- Utilizan spy para validar la implementación y no el comportamiento. **Esto es un error conceptual grave.**

Seguimiento

- Tienen chores muy poco claras, por ejemplo "Reformular modelos"
- Buen uso de features, aunque no tienen ningún bug marcado.

Nota

- 5

Grupo 6

Demo

- Cuando las contraseñas no coinciden se borran ambos campos.
- Todas las reviews son para eventos pasados, por lo que hoy son irrelevantes. Así como está la funcionalidad carece de sentido.
- ¿Eventos similares? ¿Cómo me entero de un nuevo evento relevante? ¿Avisame cuando se crea un evento que me puede gustar!

- Hoy no se puede cancelar.
- El foro no es un foro, son anuncios. La aplicación necesita aspectos sociales.
- Tampoco se puede compartir.

Código

- Los JSP tienen chequeos manuales de acceso para decidir mostrar u ocultar partes de la web. estos chequeos duplican lógica que corresponde a Spring Security, ya que necesita aplicar los mismos criterios y encima depende de magic strings. En su lugar, debieran adoptar la taglib de spring-security (`spring-security-taglibs`), que permite luego hacer cosas como `<sec:authorize url="/admin">` o `<sec:authorize access="hasRole('ADMIN')">` y dejar que Spring Security aplique las mismas reglas definidas una única vez.
- Definen un `MailService` y un `AsyncMailService` . El si los métodos son async o no es un detalle de implementación, no debiera definir a la interfaz. Esta separación sólo confunde y pone la carga cognitiva en el cliente en saber cual usar cuando.
- Los emails siempre usan el `Locale` del usuario que hizo la acción, no la del usuario que va a recibir el correo.
- Los modelos tienen fechas y horas formateadas que imprimen en los JSP *as-is*. Esto es un error, ya que el formato de fecha y hora es algo inherentemente atado a la internacionalización.
- Concatenan textos en la vista en lugar de interpolar strings.
- Imprimen valores en los JSP sin escaparlos, siendo susceptibles a XSS.
Esto es un error grave.
- Hacen uso inconsistente de boxed-primitives, con usos donde el valor `null` no parece tener sentido, y el uso de auto-unboxing es proclive a producir `NullPointerException`

Seguimiento

- Buen uso de la herramienta.

Nota

- 7

Grupo 7

Demo

- El tooltip de la ubicación no muestra nada extra. Podría quitarse.
- En lugar de invitaciones son solicitudes.
- Mostrar partidos empatados en las stats.
- Estaría bueno que lo logueen al usuario al verificarse.
- Al cambiar la contraseña pedir la contraseña actual.
- El "delete group" podría estar mas expuesto.
- Estaría bueno mostrar la cantidad de miembros por equipos en la vista de assign teams.
- Muy bueno el asignar jugadores basandose en las estadísticas.
- Algunas acciones requieren feedback, como por ejemplo guardar los cambios en los equipos, finalizar partido, invitar a alguien.
- Las estadísticas pueden estar dando información incorrecta.
- En la vista del perfil/eventos falta aclarar qué deporte es.
- Faltaría mostrar algunos datos agregados, como el resultado total de cada equipo.

Código

- Cumplen a medias con la no instanciabilidad de clases utils. Revisar item 4 de Effective Java.
- `VerifyMailController` no es un nombre adecuado, además realiza más acciones como por ejemplo reinicio de contraseña
- Manejan distinta semántica de Page en los Services y Persistence. Lo adecuado seria tener la misma semántica, y ser la capa de Persistence quien maneje los detalles de implementación (por ejemplo, restar 1 y mapear a offset propio de SQL).
- Buena cobertura de tests.

- No debiera ser necesario setear manualmente cada texto internacionalizado al armar un mail. Por ejemplo:

```
context.setVariable("welcomeTitle",  
                    messageSource.getMessage("welcome.email.title", nul
```

- En algunos casos llaman al servicio de mail con `try catch` y en otros no, sin un criterio evidente.
- El método `getSessionSport` retorna un String en lugar del modelo adecuado.
- Utilizan boxed primitives y no parece ser por diseño.
- Utilizan clases de `java.sql` fuera de la capa de persistencia. La existencia de una base de datos es un detalle de implementación del DAO, no debiera verse reflejada en webapp.

Seguimiento

- Resta 0.5 por UX (sprint 1)
- "Deuda técnica" es una chore muy poco clara.
- Buen uso de features y bugs.

Nota

- 7.5

Grupo 8

Demo

- Al inscribirme puedo inmediatamente puntuar, aunque no me hayan aceptado.
- El que las universidad sean texto libre no hace nada para evitar inconsistencias (typos o simples acrónimos: ITBA vs Instituto Tecnológico de Buenos Aires)
- El filtro de curso no maneja bien las tildes
- Filtrar por curso nunca encuentra nada

- En inglés no, pero en español todo es llamado "curso", lo que es confuso.
- El mail pidiendo el pago del curso no tiene los detalles para poder enviar la notificación de pago.

Código

- Las vistas concatenan strings en lugar de interpolarlos. **Esto es un error grave.**
- No internacionalizan todo, por ejemplo símbolos de monedas.
- Loggean en Info a STDOUT en producción. **Esto es un error conceptual grave.**
- El `LoggedInUserAdvice` es un advice, que no sólo lidia con el logged user... sino que también provee Exception Handling.
- Los `@ControllerAdvice` presentes en `ar.edu.itba.paw.webapp.advice` nunca se usan, ya que no están en un paquete escaneado por Spring.
- El no tener un advice / no exponer un puente entre auth y user los lleva a repetir muchísimo código en todos los controllers.
- Configuran simultáneamente exception resolvers y exception handlers para las mismas excepciones en lugares distintos de la aplicación.
- Implementan servicios en la capa web. Esto es una violación al modelo de capas jerárquicas establecido. **Es un error conceptual grave.**
- Los controllers validan reglas de acceso que debieran estar resueltas a nivel de Spring Security. **Esto es un error conceptual.**
- El `EmailServiceImpl` (capa de servicios) referencia a templates y sus contenidos que sólo existen en la capa webapp. Siendo una capa superior, esto es una violación al modelo de capas jerárquicas establecido. **Esto es un error conceptual grave.**
- Las clases piden que les inyecten atributos que luego no usan.
- Existen servicios que hablan en términos de vistas y hacen referencia a recursos de la web. **Esto es un error conceptual grave.**
- Eliminan a mano relaciones en lugar de usar `ON DELETE CASCADE`
- Ponen los forms en la capa de modelos, referenciando mensajes de internacionalización que sólo existen en webapp.

Seguimiento

- "Arreglar imagenes default ..." es más un bug que un chore.
- El uso de bugs y chores es prácticamente inexistente.

Nota

- 4

Grupo 9

Demo

- No hace login el cambiar contraseña
- http://pawserver.it.itba.edu.ar/paw-2025a-09/admin/dashboard/users?query=&_active=on&_inactive=on&_suspended=on tengo que hacer enter en el search para que se apliquen los filtros
- No hay confirmar contraseña
- El que acepta el trueque recibe el mail para continuar la conversación pero no le llega un CTA claro
- Aunque me suspendan la cuenta, si quedo en sesión, puedo hacer trueques y seguir usando la cuenta
- Comentarios muy largos no se cortan
- Hay un toggle de usuarios inactivos para filtrar pero no se usa ahora mismo

Código

- El archivo `schema.sql` corresponde en la capa de persistencia
- Parte de los `preauthorize` parecen ser lógica de negocio más que chequeos de seguridad
- El string `tab` debería ser un enum
- Arrojan una `RuntimeException` al no encontrar un user en `VerificationServiceImpl`, cuando un `NotFound` custom o el default son más apropiados

- Falta cobertura de casos para los métodos de servicios complejos, como por ejemplo `hideAndSwitchOwnersByTradeId` o las clases `BookInfoDaoJdbcImpl` y `BookPostDaoJdbcImpl`
- Falta un mayor uso de spring security, como por ejemplo para usuarios bloqueados.
- No escapan `%` o `_` en el `ILIKE`
- Esperaría un nivel de testing mayor para `TradeDaoJdbcImpl`. Recomendable separarlo en distintos daos
- En continuación al punto anterior, la contracara de acoplar los modelos de manera que posean una referencia a las instancias relacionadas, es que los obliga a (1) siempre traerse los otros objetos, (2) crearse sus rowmappers encadenados. En el uso actual no veo justificación para esta decisión
- Overridean `equals()` pero no `hashCode()`
- Usan `java.sql.Timestamp` en la capa models. El uso de una base de datos SQL es un detalle de implementación

Seguimiento

- Ok planificación
 - Algunas de sus chores son features en realidad, analizar si agregan valor al usuario
 - "Manejar excepciones, ahora mismo tira error." es un mensaje poco claro para una chore

Nota

- 8

Grupo 10

Demo

- Falta navbar en el detalle de un evento en particular.
<http://pawserver.it.itba.edu.ar/paw-2025a-10/events/3>. Tiene navegación persistente para ir para atras pero esto deberia sumarse a la navbar.

- Para anónimos hay una solapa en la navegación llamada Explore que te manda a a iniciar sesion. El explore es un dashboard especifico para un usuario loggeado. El explore no deberia aparecer si un usuario es anonimo.
- Extensa funcionalidad de admin y moderación. Se pueden desactivar usuarios, journeys, eventos. Se pueden agregar universidades, intereses, ciudades.
- Tienen bien definido funcionalidad de dos roles.
- Tiene validación de email y cambio de contraseña.
- Agradable funcionalidad de mails.
- Bien completo la funcionalidad de autocomplete.

Código

- El uso general del CacheManager esta bien, pero están cacheando imágenes, que al ser un ConcurrentMapCacheManager no evicta cuando se acerca al limite de memoria del sistema o por lo menos un TTL, causando que eventualmente si cachean suficientes imágenes se cae el sistema. **Esto es un error grave.**
- Try catchean una Exception en plano en NoExistingJourneyValidator. Esto indica que no están usando custom exceptions en la capa de service.
- Buen uso del AccessHelper. Sin embargo validan bloqueo de usuarios en el AccessHelper, cuando un usuario bloqueado directamente no debería poder autenticarse. Lo mas correcto es utilizar el field accountNonLocked del UserDetails y no permitir su autenticación. Pueden agarrar el LockedException si quieren mostrar una pantalla personalizada de bloqueo.
- Tienen un AttendeesLimitValidator innecesario que puede ser implementado como un @Max y @Min. Si quieren mostrar el error indicando ambos limites nombrar el validador de forma mas genérica.
- Dentro del EventController llaman una funcion llamado getEventIdByResponseld, que se utiliza para verificar si la respuesta corresponde al evento. Esto es un checkeo que no corresponde en el controller. Pasa igual en JourneyController, con deleteJourneyReplyForm.
- Hay varias instancias tanto en service como controller de `LOGGER.error("xx not found")`, sin indicar ningún tipo de valor especifico

(LOGGER.error("user not found")). Son pocos útiles.

- En ImageController hacen un try catch después de tirar un error en el propio controller. Esto es un antipattern. Deberían directamente mapear la respuesta de Optional.empty a un ResponseEntity.notFound.
- En web.xml están mapeando tanto el Exception como el Throwable a 500. Uno de los mappings es innecesario.
- ExpiredPassTokenException tiene un campo privado sin final ni setter.
- En Event tienen un field que es Optional. Esto es incorrecto ya que un field termina pudiendo ser null, Optional.empty y Optional.of. Optional solo debería ser usado para resultado de función. Esto no es una buena practica de Java.
- Tienen un getFormattedDate() en JourneyResponse que se devuelve en formato String. Esto es principalmente redundante ya que te limita a un tipo de formato que no corresponde en el modelo y deberían usar la librería de temporals en thymeleaf o fmt de JSTL
- Utilizan assertEquals del resultado de testCreateEvent en tests de service. Esto compara igualdad de identidad al no tener overrideado el equals en Event.
- Hay múltiples tests (replyToEvent, delete, update) sin Asserts en service tests. Esto indica mal diseño de contrato de la interfaz en Service.
- En service están lanzando múltiples IllegalArgumentException e IllegalStateException con mensajes custom en vez de una excepción custom.
- En DAOs lanzan RuntimeException en plano.
- Uso del StringBuilder en JdbcDaoUtils no es muy correcto cuando no hay inserciones dinámicas. Utilizar un String.format o concatenación de Strings.
- Buena cobertura de Tests en el DAO.

Seguimiento

- Buen uso de features y bugs
 - Sobreuso de chores, muchas tareas que son chores pueden ser features.
- Ejemplos

- Agregar funcionalidad de create, update, y delete en la tabla de cities en la dash
- Agregar funcionalidad de delete, update en el universities tab de la dashboard
- Bloqueo de usuarios en la admin dashboard

Nota

- 7

Grupo 11

Demo

- Agregaría una redirección del register de médico a register de paciente
- Agregaría modificación de archivos
- Hay que revisar la funcionalidad de reminder

Bueno:

- Filtros activos
- Agradable UI
- Bien los mails (l10n, CTA con botón)

Código

- Se repite `LocalDateTime.now(Zoneld.of("America/Argentina/Buenos_Aires"))`
- La inexistencia de una imagen causa que de todas formas, el endpoint asignado devuelva una por default. Tendría más sentido que se encargue de esto la vista.
- Se mezcla el uso de `optional.empty` o `id = -1` para la detección de una imagen no existente
- El método `UpdateDoctorRating(...)` no mantiene el estilo apropiado
- Buena cobertura de tests, pero falta métodos más complejos como el `checkToken(...)`

- Tendría más sentido que los static rowmappers se definan como los strings estáticos que tienen, y no al instanciar la clase

Seguimiento

- Algunos mensajes de chores deberían ser más informativos ("Email issue")

Nota

- 8

Grupo 12

Demo

- Cuando me registro no me llega ningún correo de verificación pero tampoco me deja logeado. Debería dejarme logeado si no hay mail de verificación
- Falta feedback cuando solicito recuperar contraseña.
- Cuando miro los detalles de reserva el wording de las imágenes del paquete se muestra pero no hay contenido
- Usan ingles como idioma por defecto cuando me registro. Esto hace que siempre el idioma este en ingles a menos que lo fuerce por URL
- Puedo editar las publicaciones ajenas
- Hace ruido que una vez que me convierto en wearer me siga diciendo de convertirme en wearer
- Estaría bueno poder visualizar los espacios no disponibles en el detalle de la publicación
- No puedo agregar un espacio desde la vista "Espacios" seria mejor dejarlo hacer eso en esa pagina
- Puedo hacer solicitudes a espacios inactive, falta ese chequeo falta paginar las reviews
- El admin solo puede agregar tags y tamaños, faltaría editar y eliminar

Código

- Los resources de la web deberían estar en la carpeta webapp y no en la carpeta resources de Java.
- Logean a nivel debug en producción. **Esto es un error grave.**
- No es necesario tener el `userDataSessionFilter` que almacena el current user data a nivel sesion. Este se puede obtener a partir de la autenticación. El mismo comportamiento puede ser obtenido utilizando un ModelAttribute
- Como fue marcado en la demo, falta un control de accesos a recursos a partir del ownership del user.
- Bien por utilizar custom validators
- No utilizan interpolaciones para los mensajes de error de i18n
- Poseen lógica de negocio en los controllers. **Esto es un error conceptual grave.**
- Las clases de sus modelos en "data" no son realmente modelos sino una especie de DTO. No es necesaria esta capa extra para devolver al JSP un único objeto que wrappee todo
- Hacer uso de `printStackTrace` es una mala practica, ya que logea a STDERR sin contexto. Siempre usar loggers.
- En las interfaces la declaración de que el método es publico es redundante
- Si bien usan transactional, solo lo utilizan en los métodos que son read-write, deberian tambien utilizarlo en los metodos read-only con `@Transactional(readOnly = true)`
- La cobertura de tests en servicios es muy escasa teniendo métodos que realizan bastante lógica de negocio como los create
- Esta mal utilizado el JDBC insert ya que en lugar de inicializarlo una unica vez lo inicializan en cada llamada a método. Esto no es thread safe y puede causar muchos problemas de concurrencia. **Esto es un error grave.**
- Para los tests de DAO pueden usar la anotación `@Rollback` para evitar tener que eliminar los datos luego de cada test
- Buena cobertura de Tests en el DAO.
- Utiliza `java.sql.Timestamp` en los modelos. El uso de una base de datos SQL es una cuestión de implementación. Debieran usar tipos como `LocalDate` y `LocalDateTime`

Seguimiento

- Las chores no deben tener puntos asignados
- Hay un alto uso de chores lo que parece indicar que hay muchas tareas que no son chores
- Bien por utilizar bugs
- Las stories estan bien definidas
- Resta 0.5 por entrega intermedia

Nota

- 6

Grupo 13

Demo

- El botón de filtros no funciona.
- Algunas pestañas no se remarcan al estar activas.
- El añadir jugadores anónimos debiera permitirme definir el nombre, y en el mejor de los casos, poder poner un mail para que le llegue la invitación.
- Si quiero añadir a una persona en específico no puedo hacerlo.
- Ser mas claro cuando se va a elegir el equipo contrincante.
- Los mails me llegan en español pero la pagina la veo en inglés.
- Si ya existe un usuario tiran un 500 en lugar de mostrar un mensaje apropiado.
- Es difícil darse cuenta cómo dejar una review.
- Falta feedback en situaciones como por ejemplo unirse a un partido.
- Si el owner cancela una reserva, el mail no le llega al usuario que le cancelan.
- Mandan un recordatorio cada 30 minutos el día del evento. Esto es bastante molesto.

Código

- En todas las capas de la aplicación tienen una clase App que no se utiliza.
- Pushean archivos vacíos al repositorio como `jvm.config` y `maven.config`.
- Los controller contienen lógica de negocio.
- Cumplen a medias con la no instanciabilidad de clases utils. Revisar item 4 de Effective Java.
- Manejan control de acceso en los servicios en lugar de hacerlo a través de Spring Security
- Inyectan servicios que luego no utilizan.
- No testean casos importantes como por ejemplo crear un Complex en la capa de persistence.
- No validan el estado final de la base de datos en los tests.
- Utilizan DAOs para setear las precondiciones en los tests de persistence. También utilizan métodos de la clase bajo test para setear precondiciones. En definitiva, estos tests no son unitarios. **Estos son errores conceptuales graves.**
- La cobertura de tests no es adecuada. Por ejemplo, hay DAOs completos sin testear (UserDaoImpl).
- En algunos casos utilizan boxed primitives y en otros no, para por ejemplo definir el Rating de un modelo
- EmailTemplates no es un modelo.
- Cuando overridean `equals`, no overridean `hashCode`. Ver item 11 de Effective Java.
- La capa de persistence recibe offset como parámetro. Sería más adecuado usar page para abstraer al cliente de esta lógica.
- Utilizan clases de `java.sql` fuera de la capa de persistencia. La existencia de una base de datos es un detalle de implementación del DAO, no debiera verse reflejada en webapp.

Seguimiento

- Resta 0.5 por entrega intermedia

- Buen uso de la herramienta.

Nota

- 5.5

Grupo 14

Demo

- El login no muestra errores
- Muestran el error default al registrar email invalido
- La UX de creación de user no está super intuitivo
 - Te envía un email para confirmar y no te avisa, dejando efectivamente softlockeado al user
- El register no logea
- Los cumpleaños solo se muestran de a 1 a pesar de que 2 personas cumplan el mismo día
 - Está bugeado el mensaje; el día del cumpleaños muestra que faltan 365 días.
- Las fotos default parecen estar rotas
- El flujo de las áreas de trabajo parece estar incompleto.
 - El user no puede ver a que área pertenece
 - El user no recibe contenido diferenciado por pertenecer a FRONT END o a HR
 - El user solo puede ver sus compañeros si filtra en el directorio completo de la compania por su área, pero se la tiene que saber de memoria.
- Los anuncios no se pueden leer por la página ni enviar a un subset de usuarios (solo se envían vía email)
- Los hover effects de los contenedores están de más ya que no todo el contenedor es clickeable

- Quedó un contenido dinámico sin C:OUT; son vulnerables a inyecciones XSS. **Esto es un error grave.**
- El search de users son páginas de 3 users que muestran poca data.
- La paginación no muestra cantidad de páginas dice "last".
- Bien de features. No se terminan de aprovechar los flujos por falta de conexión entre algunas de las features,
 - ej: Jefe de área comunicando a su equipo cierto anuncio,
 - poder ver futuras vacaciones de su equipo mientras decide si aceptar o no alguna.

Código

- Algunos controllers contienen lógica de negocio ej: `editEnterprise`
- El ErrorController está mal nombrado, es un ControllerAdvice
- Buena cobertura de tests en persistence, bien armados.
 - Falta cobertura de los casos no felices y excepciones.
 - La capa de servicios no tiene tests a pesar de que hay algunas funciones en esta capa no triviales.
- Buen manejo de roles
- Buen manejo de bean validation
- Utilizan boxed-primitives sin chequear correctamente si son null. Ej `roomId` , `photoId`
- Utilizan `java.sql` en capa de modelos. La existencia de una base de datos es un detalle de implementación.
- La configuración de prod de logback logea en nivel `DEBUG` cuando debería estar seteado en `WARN`
- El uso de transactional es correcto.

Seguimiento

- Ok el seguimiento ok el plane

Nota

- 8

Grupo 15

Demo

- La vista actual de tabla para elegir turnos no es muy amigable
- "Podés agregar información extra aquí::" con doble dos puntos.
- Al subir un estudio por defecto no lo ve nadie, pero el workflow no te ayuda a darte cuenta de que tenés que dar acceso. Debiera ser mucho más guiado, si subo un estudio es para que un médico lo pueda ver, sino ni lo hago en primer lugar.

Código

- Los archivos de textos para locales específicos como `en_US` heredan del archivo `en`, no es necesario copiar y pegar todas las keys si no tienen diferencias.
- En la convención de nombres de Java, todos los nombres de paquetes deben ser en minúsculas.
- El código hace uso de magic strings como "previous" o "next" que están repartidos y duplicados en múltiples puntos de la aplicación.
- Código de la forma `Assert.assertThrows(...).getMessage().contains(...)` sólo valida que se lance la excepción, no que realmente contenga el mensaje indicado.
- La `baseUrl` usada en los emails no debiera estar hardcoded en el servicio, sino inyectarse desde webapp en tiempo de ejecución. Lo mismo con el email a usar como `from`
- Imprimen valores a JSP usando `${...}` y no `<c:out>`, lo que los expone a vulnerabilidades XSS. **Esto es un error grave.**
- Los JSP tienen chequeos manuales de acceso para decidir mostrar u ocultar partes de la web. estos chequeos duplican lógica que corresponde a Spring Security, ya que necesita aplicar los mismos criterios. En su lugar, debieran adoptar la taglib de spring-security (`spring-security-taglibs`), que permite luego hacer cosas como `<sec:authorize url="/admin">` o `<sec:authorize`

`access="hasRole('ADMIN')">` y dejar que Spring Security aplique las mismas reglas definidas una única vez.

- Usan alt texts para las imágenes (lo que es positivo para accesibilidad!), pero los mismos no están internacionalizados.

Seguimiento

- Resta 0.5 por entrega intermedia

Nota

- 7.5

Grupo 16

Demo

- El link de verificación no hace autologin
- Bien por tener muchos datos cargados
- El flujo de explorar usuarios para sumar amigos me lleva a la home pero sin ningún listado para ver usuarios
- Hay una buena variedad de filtros, aunque tal vez es mejor esconder algunos porque a primer impresión parecen un poco abrumadores por la cantidad
- Falta CTA en la lista vacía cuando hago un search
- Bien por escapar los % y _
- Si busco " OR 1=1 se cuelga la app
- La home es completa y personalizada para el user
- Cree un podcast sin imagen y me retorno el form de crear podcast pero sin errores
- Estaría bueno poder ver las pagina sin estar logeado
- Sería bueno poder filtrar las actividades por tipo
- Buena cantidad de funcionalidad del user moderador

Código

- Los POM hijos no deberían tener nunca el tag version en su dependency
- Logean a nivel Debug en producción. **Esto es un error grave.**
- Bien por usar custom validators
- Como se menciona en la Demo, falta un control de accesos por ownership del recurso para evitar que cualquier user autenticado pueda realizar acciones sobre recursos que no le pertenecen
- el método `getCurrentUser` no es necesario ya que se puede obtener el mismo resultado con un `@ModelAttribute` en un controller advice
- Realizan validaciones que deberían ser parte de Spring Security en el controller

```
boolean isUserCreator = userCreatorId != null && userCreatorId == creatorId; if (!isUserCreator) return new ModelAndView("redirect:/creator/" + creatorId);
```
- Utiliza `java.sql.Timestamp` en los modelos para convertir un Instant a Date. El uso de una base de datos SQL es una cuestión de implementación. Deberían usar tipos como `LocalDate` y `LocalDateTime`
- Omiten modificadores de acceso y no parece ser por diseño
- Utilizan `jdbcTemplate.update` para insertar datos en lugar de usar el `jdbcTemplate.insert`, sería más sencillo utilizar el `jdbcTemplate.insert`
- Existen Row mappers que no son `private static final` y son instanciados en cada ejecución
- Para los tests de DAO pueden usar la anotación `@Rollback` para evitar tener que eliminar los datos luego de cada test
- Buena cobertura de Tests en el DAO.
- En los tests hacen verificaciones con `Mockito.verify()` y `Mockito.never()`. Esto valida la implementación en lugar del comportamiento. **Esto es un error conceptual grave.**
- En los services hacen inyección de dependencia por medio de constructor y de setters, esto es inconsistente y debería ser realizado de la misma forma a lo largo de toda la app.
- No utilizan `@Transactional` en ningún lugar de la aplicación.

Seguimiento

- Las stories estan bien definidas
- Bien por utilizar bugs
- Algunas stories no tienen estimacion

Nota

- 6

Grupo 17

Demo

- Muy completo en funcionalidad
- Tiene validación de email y cambio de contraseña
- No tienen roles pero si tienen Ownership sobre torneos y sesiones
- Mails muy completos y agradables
- Tienen botones y dropdown sin i18n <http://pawserver.it.itba.edu.ar/paw-2025a-17/sessions> en el dropdown de generos.
- Al explorar las sesiones para buscar nuevos juegos te siguen mostrando las sesiones que estan llenas. Dejan de mostrar la cantidad de jugadores / cantidad total (ejemplo, muestran 2 en vez de 2/2) haciendo que sea difícil de ver que esta lleno la sesión.
- Pagina de detalle de sesión no tiene favicon <http://pawserver.it.itba.edu.ar/paw-2025a-17/sessions/52/detail>
- Al invitarte a una sesión te llega un mail pero el CTA apunta a algo que ya no existe. Debería enviarte al detalle de la sesión.
- Al intentar editar un torneo del cual no sos dueño tira 404. Debería tirar 403.

Código

- Dentro de los jsps utilizan multiples veces `${pageContext.request.contextPath}` en vez de `<c:url>`.

- Tienen archivos .sql en webapp. Estos tienen que ir en los resources de persistence.
- En configure de WebSecurity ignoran las rutas de 403 y 404, pero posteriormente los incluyen al configurar el HttpSecurity.
- Manualmente manejan el Accept-Language en los endpoints del controller en vez de delegarle a Spring el parseo de Locale con `LocaleContextHolder`
- En newEditProfile están guardando la imagen, actualizando el nombre o editando los juegos favoritos. Esto es lógica de negocio que debería ir en el service. Pasa lo mismo en uploadGameImage. **Esto es un error conceptual grave.**
- En tournamentStart del controller estan pidiendo por el Tournament y después comenzando el tournamentId como validacion. Esto es logica de negocio que no corresponde en controller. Tambien realizan validacion de NotAnAdmin en vez de utilizar un AccessControl. Pasa similarmente en setTournamentSessionWinner. Pasa similarmente en newEditProfile para guardar imágenes.
- Nunca validan existencia de ids de modelos existentes, como gameId a la hora de crear sesión. Tampoco catchean ForeignKeyException en el DAO generando un 500 en vez de devolver un StatusCode 400.
- Varias funciones en GameServiceImpl sin `@Transactional(readonly=true)` (GameServiceImpl.getAllGames, getAllTeamSizes)
- Varias funciones en TournamentServiceImpl sin `@Transactional` (startTournament, updateTournament)
- Hay unit tests que no son unitarios, validando cambios de estados de objetos mockeados no devueltos por la función en si. Ejemplo es el addParticipantToEncounter. **Esto es un error conceptual grave.**
- Falta cobertura de tests para casos de errores que se lanzan en service, como addParticipantToEncounter que no se valida cuando se llega al limite de participantes de un encounter. Pasa similar en UserServiceImplTest.
- Borran los usuarios que expiraron y no verificaron, en vez de proveer la posibilidad de regenerar el token de verificación.
- Tests en DAO, falta cobertura para findBys de SessionJdbcDao, que validen posibles combinación de filtros y paginación. Estos son queries no triviales que deberían tener amplia cobertura.

- Rankings para un juego se están obteniendo del GameJdbcDao, accediendo principalmente a la tabla de sessions y teams que no corresponde para el GameJdbcDao. Debería ser responsabilidad de SessionJdbcDao
- En TournamentJdbcDao.getRound para armar la mapa de encounters utilizan un RowCallbackHandler con un closure sobre encounterMap. Este no es una forma correcta de extraer rows de un result set ya que depende de un dato externo al RowCallbackHandler. Para estos casos utilizar un ResultSetExtractor.

Seguimiento

- Buena definición y criterio para el uso de features, bugs y chores
- Faltan features estimadas

Nota

- 8